

// Jivops

PostgreSQL

Guía completa — 30 lecciones

Fundamentos · SQL e índices · Rendimiento y mantenimiento · Alta disponibilidad · Backup y recuperación · Seguridad y producción

Contenido

Módulo 1: Fundamentos de PostgreSQL

Lección 01 — Qué es PostgreSQL y cómo gestiona las conexiones

Lección 02 — MVCC — concurrencia multiversión

Lección 03 — Tipos de datos y extensibilidad

Lección 04 — Esquemas, roles y search_path

Lección 05 — Instalación y estructura del directorio de datos

Módulo 2: SQL, índices y planes de ejecución

Lección 06 — Transacciones y niveles de aislamiento

Lección 07 — Índices B-tree y cuándo se usan

Lección 08 — Otros índices — GIN, GiST, BRIN, parciales y de expresión

Lección 09 — EXPLAIN y EXPLAIN ANALYZE

Lección 10 — Restricciones e integridad de datos

Módulo 3: Rendimiento y mantenimiento

Lección 11 — VACUUM y el bloat

Lección 12 — Autovacuum y su ajuste

Lección 13 — Estadísticas, ANALYZE y el planificador

Lección 14 — Memoria — shared_buffers, work_mem y effective_cache_size

Lección 15 — Connection pooling con PgBouncer

Módulo 4: Alta disponibilidad y replicación

Lección 16 — WAL y checkpoints

Lección 17 — Replicación en streaming

Lección 18 — Replicación síncrona frente a asíncrona

Lección 19 — Replicación lógica

Lección 20 — Failover y Patroni

Módulo 5: Backup, recuperación y actualizaciones

Lección 21 — Backup lógico (pg_dump) frente a backup físico

Lección 22 — Archivado de WAL y recuperación a un punto en el tiempo (PITR)

Lección 23 — Herramientas de backup — pgBackRest y Barman

Lección 24 — Probar las restauraciones — RTO y RPO

Lección 25 — Actualizaciones menores y mayores — pg_upgrade

Módulo 6: Seguridad y producción

Lección 26 — Autenticación y `pg_hba.conf`

Lección 27 — Roles, privilegios y seguridad a nivel de fila (RLS)

Lección 28 — TLS y cifrado

Lección 29 — Monitoreo — `pg_stat_activity`, `pg_stat_statements` y bloqueos

Lección 30 — Proyecto final: PostgreSQL listo para producción

Módulo 1: Fundamentos de PostgreSQL

Lección 01 — Qué es PostgreSQL y cómo gestiona las conexiones

Un proceso principal (postmaster) escucha y crea un proceso de servidor propio por cada conexión, por lo que muchas conexiones simultáneas consumen memoria y CPU y suele convenir limitarlas o usar un pool.

Cada conexión es un proceso del sistema operativo con su propia memoria; aunque estén inactivas, miles de procesos consumen recursos y aumentan la contención, mientras que un pool reutiliza pocas conexiones reales.

Lección 02 — MVCC — concurrencia multiversión

Que cada transacción vea una instantánea consistente: las lecturas no bloquean a las escrituras ni al revés, porque una actualización crea una versión nueva de la fila y deja la anterior visible para las transacciones que aún la necesitan.

Las versiones antiguas quedan como tuplas muertas ocupando espacio cuando ninguna transacción las necesita; sin limpieza, tablas e índices crecen con datos obsoletos, y VACUUM marca ese espacio como reutilizable.

Lección 03 — Tipos de datos y extensibilidad

Además de los tipos habituales ofrece jsonb (JSON binario indexable), arrays, rangos y tipos propios, y se amplía con extensiones (como pg_stat_statements o PostGIS) que añaden funciones, tipos e índices.

Permite consultar dentro del documento sin un esquema fijo y de forma eficiente gracias al índice, combinando la flexibilidad de los documentos con las garantías transaccionales de una base relacional.

Lección 04 — Esquemas, roles y search_path

Un esquema agrupa objetos dentro de una base de datos y search_path determina en qué esquemas y en qué orden se buscan los nombres sin calificar; los roles son usuarios o grupos a los que se conceden privilegios.

Un nombre sin calificar depende del search_path, que puede cambiar por sesión o rol; un objeto con el mismo nombre en otro esquema podría usarse por error o suplantarse, y calificar el nombre elimina esa ambigüedad.

Lección 05 — Instalación y estructura del directorio de datos

PGDATA contiene la configuración, los archivos de las tablas y el WAL en pg_wal; postgresql.conf fija los parámetros del servidor y pg_hba.conf controla quién puede conectarse y cómo se autentica.

Cada commit debe escribir y sincronizar el WAL, así que su latencia afecta al tiempo de las transacciones; además se escribe de forma secuencial y no debe competir con el acceso aleatorio a los archivos de datos.

Módulo 2: SQL, índices y planes de ejecución

Lección 06 — Transacciones y niveles de aislamiento

Por defecto usa READ COMMITTED, donde cada sentencia ve lo confirmado hasta ese momento; REPEATABLE READ mantiene una única instantánea durante la transacción, y SERIALIZABLE evita anomalías pero puede abortar transacciones, que la aplicación debe reintentar.

PostgreSQL puede abortar una transacción con un error de serialización para evitar una anomalía; no es un fallo de la consulta sino un conflicto de concurrencia que suele resolverse repitiéndola, y sin reintentos el usuario vería errores intermitentes.

Lección 07 — Índices B-tree y cuándo se usan

Es el tipo por defecto y acelera igualdades, rangos y ordenaciones sobre las columnas indexadas; a cambio ocupa espacio y ralentiza las escrituras, porque cada INSERT, UPDATE o DELETE debe mantenerlo.

Si la consulta devuelve una gran parte de la tabla, leer secuencialmente es más barato que saltar entre índice y tabla; el planificador elige según costes estimados, no por la mera existencia del índice.

Lección 08 — Otros índices — GIN, GiST, BRIN, parciales y de expresión

GIN indexa valores compuestos como jsonb, arrays o texto completo; GiST sirve para datos geométricos y rangos; BRIN es muy pequeño y útil en tablas enormes con orden natural; parciales y de expresión indexan solo un subconjunto de filas o un valor calculado.

Solo indexa las filas que cumplen la condición, así que es mucho más pequeño, cabe mejor en memoria y cuesta menos mantenerlo; si las consultas siempre filtran por ese estado, contiene justo lo necesario.

Lección 09 — EXPLAIN y EXPLAIN ANALYZE

EXPLAIN muestra el plan estimado sin ejecutar la consulta; EXPLAIN ANALYZE la ejecuta de verdad y añade tiempos y filas reales, lo que permite comparar estimación y realidad; con INSERT, UPDATE o DELETE conviene usarlo dentro de una transacción que se revierta.

El planificador elige el plan según sus estimaciones, y si son erróneas puede escoger una mala estrategia; suele indicar estadísticas desactualizadas o correlaciones que desconoce, y se corrige con ANALYZE o estadísticas ampliadas.

Lección 10 — Restricciones e integridad de datos

Las restricciones (NOT NULL, UNIQUE, CHECK, claves primarias y foráneas) hacen cumplir la integridad en la propia base de datos; PostgreSQL no crea automáticamente un índice en la columna que referencia una clave foránea, por lo que suele convenir crearlo.

Para comprobar la integridad hay que buscar las filas hijas que referencian la fila borrada; sin índice, esa búsqueda recorre toda la tabla hija en cada borrado.

Módulo 3: Rendimiento y mantenimiento

Lección 11 — VACUUM y el bloat

VACUUM recupera el espacio de las tuplas muertas para que pueda reutilizarse, sin bloquear la tabla, pero normalmente no devuelve espacio al sistema operativo; VACUUM FULL reescribe la tabla y sí lo devuelve, pero la bloquea con un candado exclusivo.

La transacción antigua podría necesitar aún esas versiones de las filas, así que VACUUM no puede eliminarlas; el horizonte de visibilidad queda retenido por la transacción más antigua y el bloat crece mientras siga abierta.

Lección 12 — Autovacuum y su ajuste

Se lanza cuando las tuplas muertas superan un umbral más un porcentaje de la tabla (por defecto el 20 %); en tablas muy grandes ese porcentaje es demasiado alto, por lo que suele ajustarse por tabla para que actúe antes.

Las tuplas muertas se acumularían y tablas e índices crecerían degradando el rendimiento, además de arriesgar problemas con el agotamiento de identificadores de transacción; es mejor ajustar su agresividad que eliminarlo.

Lección 13 — Estadísticas, ANALYZE y el planificador

Describen la distribución de los datos y el planificador las usa para estimar cuántas filas devolverá cada operación; con estadísticas desactualizadas (por ejemplo tras una carga masiva) puede elegir planes malos hasta que se actualicen.

El autovacuum puede tardar en detectar el cambio y, mientras tanto, el planificador cree que la tabla es pequeña o tiene otra distribución; analizar enseguida evita planes lentos justo después de la carga.

Lección 14 — Memoria — `shared_buffers`, `work_mem` y `effective_cache_size`

`shared_buffers` es la caché de páginas del servidor (un punto de partida habitual es cerca del 25 % de la RAM); `work_mem` se aplica por cada operación de ordenación o hash, así que puede multiplicarse entre consultas y conexiones; `effective_cache_size` es solo una pista para el planificador.

Cada operación de ordenación o hash puede usar ese límite y una consulta compleja usa varias; el consumo total se multiplica por las conexiones activas, así que hay que dimensionarlo pensando en la concurrencia.

Lección 15 — Connection pooling con PgBouncer

Mantiene un número reducido de conexiones reales con PostgreSQL y las reparte entre muchos clientes; en modo transaction la conexión se asigna solo mientras dura cada transacción, aunque algunas funciones ligadas a la sesión pueden no comportarse como se espera.

La mayoría de los clientes pasa tiempo inactiva entre transacciones y no necesita una conexión real en ese momento; la conexión se presta solo mientras dura la transacción y luego la usa otro cliente.

Módulo 4: Alta disponibilidad y replicación

Lección 16 — WAL y checkpoints

Los cambios se registran primero en el WAL, de modo que tras un fallo el servidor puede rehacerlos y recuperar un estado consistente; un checkpoint vuelca a los archivos de datos las páginas modificadas, acotando cuánto WAL hay que reproducir al recuperarse.

El registro secuencial es barato de sincronizar a disco al confirmar; las páginas de datos pueden escribirse más tarde porque el WAL permite rehacer los cambios tras un fallo, así que hay durabilidad sin pagar escrituras aleatorias en cada commit.

Lección 17 — Replicación en streaming

Una réplica física recibe el flujo de WAL del primario y lo reproduce, manteniendo una copia binaria idéntica de todo el cluster; puede servir consultas de solo lectura (hot standby), debe usar la misma versión mayor, y los slots de replicación evitan que el primario descarte WAL que la réplica aún no ha recibido.

El slot obliga al primario a conservar todo el WAL que la réplica aún no ha consumido; si la réplica deja de conectarse, ese WAL se acumula sin límite, así que hay que monitorizar los slots y limitar la retención.

Lección 18 — Replicación síncrona frente a asíncrona

En la asíncrona el commit no espera a la réplica: baja latencia pero posible pérdida de las últimas transacciones en un failover; en la síncrona el commit espera su confirmación (sin pérdida de datos), a costa de latencia y de que las escrituras se bloqueen si no hay réplica síncrona disponible.

Para no perder datos el commit espera a la réplica, y si esta no responde las escrituras quedan bloqueadas; más durabilidad significa más dependencia de que la réplica esté disponible.

Lección 19 — Replicación lógica

Usa publicaciones y suscripciones para replicar cambios de filas de tablas concretas, y permite replicar entre versiones mayores distintas o hacia otros sistemas; no replica automáticamente los cambios de esquema (DDL), que hay que aplicar en ambos lados.

Permite mantener una copia en la versión nueva sincronizada mientras el sistema antiguo sigue en servicio; al final solo hay que redirigir las aplicaciones tras una breve parada, algo que la replicación física no permite entre versiones mayores.

Lección 20 — Failover y Patroni

Se apoya en un almacén de consenso (etcd, Consul...) para elegir un único líder y promover una réplica si el primario falla, evitando el split-brain, es decir, dos nodos aceptando escrituras a la vez y divergiendo.

Si el antiguo primario sigue activo habría dos nodos aceptando escrituras y los datos divergirían; reconciliar dos historias distintas es muy difícil y puede costar datos, por eso se usa un líder único con vencimiento y se aísla al nodo anterior.

Módulo 5: Backup, recuperación y actualizaciones

Lección 21 — Backup lógico (pg_dump) frente a backup físico

pg_dump genera una copia lógica de una base de datos que permite restaurar objetos concretos o migrar entre versiones, pero es lento en bases grandes y no ofrece recuperación a un instante; el backup físico copia los archivos del cluster y, junto al WAL archivado, permite recuperación a un punto en el tiempo.

Solo recupera el estado del momento del volcado y se pierden las transacciones posteriores, además de tardar mucho en hacerse y restaurarse; la recuperación a un punto en el tiempo requiere backup físico más WAL archivado.

Lección 22 — Archivado de WAL y recuperación a un punto en el tiempo (PITR)

Con el archivado de WAL activado y un backup base, permite restaurar el cluster y reproducir el WAL hasta un instante concreto (por ejemplo, justo antes de un borrado accidental) mediante un objetivo como recovery_target_time.

Si faltan segmentos de WAL la cadena se rompe y no se puede recuperar más allá del último backup; el problema se descubre justo cuando hace falta restaurar, por eso hay que monitorizar el archivado y probar la restauración.

Lección 23 — Herramientas de backup — pgBackRest y Barman

Gestionan backups completos e incrementales, el archivado de WAL, la compresión, la retención y la verificación, simplificando el PITR frente a montarlo a mano con scripts.

Define cuánto tiempo y cuántas copias se conservan, equilibrando capacidad de recuperación y espacio; sin ella, las copias se acumulan hasta llenar el almacenamiento o se borran sin criterio.

Lección 24 — Probar las restauraciones — RTO y RPO

Un backup no está validado hasta que se restaura: probarlo periódicamente en otro entorno comprueba que copias y WAL son utilizables y permite medir el tiempo real de recuperación (RTO) y la pérdida máxima de datos (RPO).

En un incidente real no hay margen para investigar por qué la copia no funciona y el servicio sigue caído; una prueba previa permite corregir procedimientos y copias con calma.

Lección 25 — Actualizaciones menores y mayores — pg_upgrade

Las menores (por ejemplo 16.3 a 16.4) solo sustituyen binarios y se reinicia el servidor; las mayores (16 a 17) cambian el formato interno y requieren pg_upgrade, volcado y restauración o replicación lógica, siempre tras ensayarlo en un entorno de pruebas.

Permite detectar incompatibilidades de extensiones, consultas o configuración antes de afectar al servicio real, medir el tiempo de parada y probar un plan de vuelta atrás si algo falla.

Módulo 6: Seguridad y producción

Lección 26 — Autenticación y `pg_hba.conf`

Se evalúa de arriba abajo y se aplica la primera regla que coincide con el tipo de conexión, base de datos, usuario y dirección; para contraseñas se recomienda `scram-sha-256`, y el método `trust`, que no pide credenciales, debe evitarse fuera de pruebas.

Se aplica la primera regla que coincide y las siguientes ya no se miran; una regla amplia colocada antes puede anular una más restrictiva, abriendo accesos no previstos o bloqueando los legítimos.

Lección 27 — Roles, privilegios y seguridad a nivel de fila (RLS)

Con roles de grupo a los que se conceden privilegios y a los que pertenecen los usuarios; la seguridad a nivel de fila (RLS) añade políticas que limitan qué filas ve o modifica cada rol, aunque el propietario de la tabla y los superusuarios la eluden salvo que se fuerce.

Una inyección SQL o una filtración de credenciales daría control total sobre la base de datos; un rol con privilegios mínimos limitaría el daño a los objetos que necesita, mientras que el superusuario evita todas las comprobaciones.

Lección 28 — TLS y cifrado

TLS cifra la conexión entre cliente y servidor; con `sslmode=verify-full` el cliente además valida el certificado y el nombre del servidor, protegiendo frente a suplantación, mientras que `sslmode=require` cifra pero no verifica la identidad; PostgreSQL no cifra los datos en disco por sí solo, para eso se usa cifrado de disco o de sistema de archivos.

El atacante puede presentar su propio certificado y el cliente lo aceptaría porque no comprueba la identidad; el canal queda cifrado, pero con el atacante y no con el servidor legítimo.

Lección 29 — Monitoreo — `pg_stat_activity`, `pg_stat_statements` y bloqueos

`pg_stat_activity` muestra sesiones y esperas, `pg_stat_statements` (extensión) agrega estadísticas por consulta para encontrar las más costosas, y `pg_stat_replication` y `pg_locks` ayudan con el retraso de réplicas y los bloqueos; conviene vigilar también las sesiones `idle in transaction`.

Mantiene bloqueos y una transacción abierta que retiene el horizonte de visibilidad, impidiendo limpiar tuplas muertas y pudiendo bloquear a otras sesiones; se mitiga con un tiempo máximo para esas sesiones.

Lección 30 — Proyecto final: PostgreSQL listo para producción

Réplicas con failover gestionado y réplicas síncronas donde la durabilidad lo exija, backups físicos con WAL archivado y restauraciones probadas, pooling de conexiones, autovacuum y memoria ajustados, TLS con `scram` y `pg_hba.conf` restrictivo, y monitoreo con `pg_stat_statements` y alertas.

Intenta mencionar al menos dos pilares: réplicas con failover gestionado, backups con WAL archivado y restauraciones probadas, pooling y ajuste de autovacuum, TLS con `scram` y `pg_hba` restrictivo, o monitoreo con alertas.